

DHRCL: Training Code LLMs with Dense Hierarchical Rewards and Curriculum Learning

Shuhang Wang, Ziming Li, Hui Cheng

Abstract

Reinforcement learning is a natural post-training paradigm for code-oriented large language models because generated programs can be evaluated through parsing, execution, unit tests, and structural analysis. However, existing methods often rely on sparse outcome rewards or statically combine heterogeneous dense signals, even though syntax validity, executability, functional correctness, and structural organization describe different and progressively dependent programming capabilities. We propose **DHRCL**, a reinforcement learning framework with Dense Hierarchical Rewards and Curriculum Learning. DHRCL decomposes feedback into syntax validation, execution success, unit-test pass rate, and AST-based structural similarity, and organizes these signals through a three-stage Syntax, Execution, Pass & Structural curriculum. Stage duration is determined automatically from recent validation trends rather than manually specified capability thresholds. We further introduce stage-aware probability-based token credit redistribution. The mechanism follows a consolidation-to-refinement principle: it emphasizes established token patterns during syntax-oriented optimization, applies uniform propagation for non-local execution feedback, and allocates more credit or blame to less-established token decisions during final functional optimization. Under a unified Qwen3-8B and KodCode protocol, the experiments compare DHRCL with binary, pass-rate, reward-model-based, and verifiable dense-reward baselines. We further evaluate DHRCL across Qwen3-4B, Qwen3-8B, and Qwen3-14B backbones, showing that its advantage remains consistent as model capacity increases.

Introduction

Large language models (LLMs) have made code generation a central application of modern language models (Chen et al. 2021; Rozière et al. 2023), while realistic benchmarks increasingly emphasize execution-based validation (Zhuo et al. 2025; Wang et al. 2023). Code is well suited to reinforcement learning (RL) because generated programs admit automatically verifiable feedback from parsing, execution, tests, and structural analysis. Prior work exploits compiler and unit-test outcomes (Le et al. 2022; Shojaei et al. 2023; Liu et al. 2023b), but sparse rewards provide little guidance when most

samples fail, while static combinations of dense rewards ignore the prerequisite relations among syntax validity, executability, and functional correctness.

We propose **DHRCL**, a code-LLM RL framework with **Dense Hierarchical Rewards and Curriculum Learning**. DHRCL combines syntax validation, execution success, unit-test pass rate, and AST-based structural similarity in a Syntax, Execution, Pass & Structural curriculum. Stage duration is selected automatically from held-out validation trends rather than manually chosen capability thresholds. DHRCL also redistributes trajectory-level credit across tokens according to a consolidation-to-refinement principle: confidence-oriented weighting in the syntax stage, uniform weighting for non-local execution feedback, and uncertainty-oriented weighting in the final stage.

Our contributions are:

- A three-stage hierarchical reward curriculum coordinating syntax, execution, functional, and structural feedback while retaining functional correctness as the primary objective.
- Trend-based automatic stage progression that adapts stage duration without fixed syntax or execution thresholds.
- Stage-aware token credit redistribution with controlled ablations across reward composition, curriculum scheduling, and token weighting.

Related Work

RL-Based Optimization for Code LLMs

RL has become a prominent post-training paradigm for aligning code LLMs with sequence-level functional correctness (Wang et al. 2024; Jiang et al. 2026). Early policy-gradient methods such as REINFORCE (Williams 1992) enable optimization beyond token likelihood, and have been adapted to code generation through actor-critic and PPO-style frameworks. CodeRL (Le et al. 2022) uses execution feedback in an actor-critic setting, while PPOCoder (Shojaei et al. 2023), RLTF (Liu et al. 2023b), and RLEF (Gehring et al. 2025) extend PPO (Schulman et al. 2017) to compiler-guided, online, and iterative feedback scenarios. Related preference- and group-based methods further improve scalable post-training: DPO (Rafailov et al. 2023) and Focused-DPO (Zhang et al. 2025b) avoid explicit critic learning, while GRPO (Shao et al. 2024; DeepSeek-AI 2025), DAPO (Yu

et al. 2025), and GSPO (Zheng et al. 2025) improve critic-free or sequence-level policy optimization. These methods strengthen the RL optimization backbone, whereas our work focuses on how code-specific rewards are constructed and organized.

Code-Specific Reward Design

Reward design is central to RL-based code generation because programs can be verified through compilation, execution, tests, and structural analysis. Existing rewards include verifiable execution or compiler feedback (Le et al. 2022; Shojaei et al. 2023; Liu et al. 2023b; Gehring et al. 2025), structured or dense signals such as AST/DFG consistency, partial test success, and task-specific code similarity (Zhao et al. 2025; Wang et al. 2026b; Wu et al. 2026), as well as learned reward models and process-level supervision (Zeng et al. 2025; Dai et al. 2024; Ye et al. 2025). Recent preference- and test-construction methods further refine supervision through test-based preferences or adversarial test generation (Zhang et al. 2024, 2025a; Wu et al. 2025; Wang et al. 2026a). Overall, these studies enrich code rewards with verifiable, dense, local, or adaptive feedback, but heterogeneous objectives are usually combined statically. This leaves stage-wise reward scheduling across syntax, execution, tests, and structure underexplored.

Curriculum Learning for Code LLMs

Curriculum learning organizes optimization from easier to harder learning units. In code generation, prior work has explored curricula over completion subtasks, data complexity, code blocks, and test cases. StepCoder (Dou et al. 2024) decomposes full program synthesis into code-completion subtasks with fine-grained optimization; data-level curricula order training samples by code complexity to improve small code LMs (Nair et al. 2024); AST-guided block splitting improves fill-in-the-middle DPO alignment (Ren et al. 2025); and TAROT (Park et al. 2026) constructs tiered test suites with capability-adaptive curriculum policies. These methods mainly schedule training units rather than heterogeneous reward objectives. In contrast, DHRCL organizes syntax, execution, functional, and structural rewards into a stage-aware curriculum and further propagates them with stage-aware token credit redistribution.

Method

We propose **DHRCL**, a reinforcement learning framework for code-oriented LLM post-training. As shown in Figure 1, DHRCL contains two coupled components. First, a dense hierarchical reward decomposes code feedback into syntax validity, execution validity, partial functional correctness, and AST-based structural alignment. Second, a stage-wise curriculum changes the primary optimization emphasis according to the policy’s current bottleneck, while probability-aware token credit redistribution controls how each trajectory-level update is allocated across generated tokens. The predefined capability order is Syntax, Execution, Pass & Structural, whereas each stage duration is determined automatically from held-out validation trends.

Dense Hierarchical Reward

Outcome-level rewards such as binary test success or aggregate pass rate often cannot distinguish locally malformed code, runtime failures, and partially correct executable programs. DHRCL therefore combines four complementary reward components. The first three directly describe program validity and functional behavior. The fourth is an auxiliary, reference-conditioned structural signal; it is not treated as semantic equivalence or as a complete measure of code quality.

Syntax Validation Rate We use a parser-based syntax reward to provide partial feedback for locally well-formed code. Given a generated program y , an error-tolerant parser extracts syntactic units $\mathcal{B}(y) = \{b_i\}_{i=1}^m$. A unit is valid if its subtree contains no parser error node, and the reward is

$$r_{\text{syntax}}(y) = \frac{\sum_{i=1}^m w_i v_i}{\sum_{i=1}^m w_i},$$

where v_i is the validity indicator and w_i is the token length. We set $r_{\text{syntax}}(y) = 0$ if no unit is extracted. Implementation details are provided in the Supplementary Document, Section “Implementation Details of Syntax Reward.”

Execution Success While syntax validity ensures that a program can be parsed at least locally, it does not guarantee successful execution. Generated code may still trigger runtime errors, exceed time limits, or cause memory-related failures. To capture runtime validity, we introduce an execution success reward:

$$r_{\text{exec}} = \mathbb{I}(\text{execution success}),$$

where $\mathbb{I}(\cdot)$ denotes the indicator function. The reward is 1 if the generated program executes successfully under the evaluation environment, and 0 otherwise. This component encourages the model to move beyond syntactic well-formedness and learn programs with valid runtime behavior.

Pass Rate The final goal of code generation is to solve the programming problem, which is typically evaluated by unit tests. Instead of using a binary all-or-nothing reward, we use the fraction of passed test cases as a dense functional correctness signal:

$$r_{\text{pass}} = \frac{n_{\text{pass}}}{n_{\text{test}}},$$

where n_{pass} is the number of passed test cases and n_{test} is the total number of test cases. This reward differentiates partially correct solutions and provides a smoother optimization signal than binary test success.

AST-Based Structural Similarity Unit-test feedback evaluates functional behavior but does not distinguish the structural organization of generated programs. We therefore introduce an auxiliary reference-conditioned AST reward. For a generated program y and a verified reference solution y^{ref} , let $\mathcal{N}(\text{AST}(\cdot))$ denote the structural normalization used by our implementation. For programs that can be strictly parsed, we compute

$$r_{\text{AST}}(y) = \text{Sim}_{\text{AST}}(\mathcal{N}(\text{AST}(y)), \mathcal{N}(\text{AST}(y^{\text{ref}}))),$$

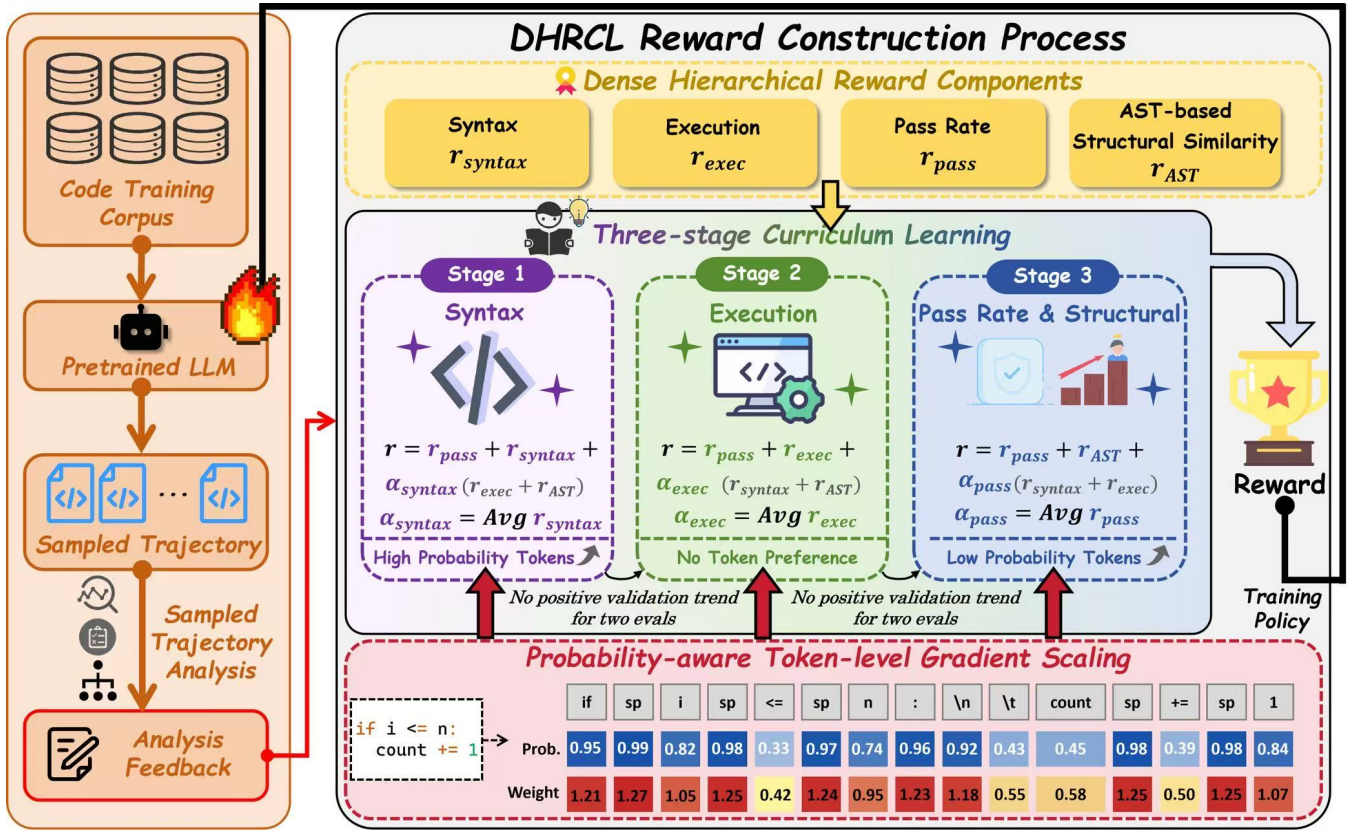


Figure 1: Overall pipeline of DHRCL.

where $\text{Sim}_{AST} \in [0, 1]$ is the implemented AST-similarity function. If the generated program cannot be strictly parsed, we set $r_{AST}(y) = 0$. Implementation details of the AST structural reward are provided in the Supplementary Document, Section “Implementation Details of the AST Structural Reward.”

AST similarity is not interpreted as semantic equivalence, maintainability, or computational efficiency. Instead, it supplies a weak structural prior derived from a verified implementation. Functional correctness remains directly supervised by the unit-test pass-rate reward, while the AST signal encourages exploration of structurally coherent implementation patterns. The reward-component ablation compares the full formulation against an otherwise identical variant without r_{AST} , thereby isolating its contribution to Pass@1, structural similarity, and the observed characteristics of accepted outputs.

Curriculum Learning for Code Generation

Although the above rewards provide dense supervision, combining them with fixed weights ignores prerequisite relations among programming capabilities. DHRCL uses three consecutive stages: Syntax, Execution, and Pass & Structural. The curriculum changes the primary optimization emphasis rather than completely enabling or disabling a reward. In particular, r_{pass} remains active throughout training because functional correctness is always the final task objective, while

previously introduced validity signals remain auxiliary components in later stages.

Syntax Stage For the i -th sampled trajectory, the Syntax-stage reward is

$$r_i = r_{pass,i} + r_{syntax,i} + \alpha_{syntax} r_{exec,i} + \alpha_{syntax} r_{AST,i},$$

where

$$\alpha_{syntax} = \frac{1}{n} \sum_{i=1}^n r_{syntax,i}.$$

Here, n is the number of trajectories in the rollout batch. The batch-level coefficient acts as a competence-dependent gate. When syntax validity is weak, downstream execution and structural signals receive less weight; as syntax validity improves, they are introduced more strongly. The pass-rate reward is retained without attenuation.

Execution Stage After syntax-oriented optimization, the primary emphasis shifts to executable behavior:

$$r_i = r_{pass,i} + r_{exec,i} + \alpha_{exec} r_{syntax,i} + \alpha_{exec} r_{AST,i},$$

with

$$\alpha_{exec} = \frac{1}{n} \sum_{i=1}^n r_{exec,i}.$$

Runtime validity often depends on interactions among definitions, control flow, data types, API calls, and memory or time

constraints. This stage therefore prioritizes whole-program execution feedback while retaining syntax and structural signals as auxiliary terms.

Pass & Structural Stage In the final stage, the primary objective combines functional correctness with the auxiliary structural prior:

$$r_i = r_{\text{pass},i} + r_{\text{AST},i} + \alpha_{\text{pass}} r_{\text{syntax},i} + \alpha_{\text{pass}} r_{\text{exec},i},$$

where

$$\alpha_{\text{pass}} = \frac{1}{n} \sum_{i=1}^n r_{\text{pass},i}.$$

The pass-rate term is the direct functional objective. The AST term is an auxiliary reference-conditioned signal and does not replace execution-based correctness. Final checkpoint selection is based on validation Pass@1, with AST similarity used only as a tie-breaker between checkpoints with identical validation Pass@1.

Trend-Based Automatic Stage Progression Fixed absolute thresholds can immediately advance a strong initialization or indefinitely retain a weak one, even when both policies exhibit meaningful learning progress. DHRCL instead determines stage transitions from the recent trend of the primary held-out validation metric. Let $m_s^{(k)}$ be the primary validation metric of stage s at the k -th evaluation, where k indexes validation observations:

$$m_s^{(k)} = \begin{cases} r_{\text{syntax}}^{\text{val},(k)}, & s = \text{Syntax}, \\ r_{\text{exec}}^{\text{val},(k)}, & s = \text{Execution}, \\ \text{Pass@1}_{\text{val}}^{(k)}, & s = \text{Pass \& Structural}. \end{cases}$$

After at least 12 validation observations have been collected in the current stage, we use the most recent eight observations,

$$\mathcal{H}_s^{(k)} = \{m_s^{(k-7)}, \dots, m_s^{(k)}\},$$

and fit the linear model

$$m_s^{(t)} = a_s + b_s t + \epsilon_t.$$

We perform a one-sided test of

$$H_0 : b_s \leq 0 \quad \text{against} \quad H_1 : b_s > 0.$$

The policy remains in the current stage while the recent window supports a positive trend. A transition is triggered only after the positive-trend test fails at two consecutive evaluations, which reduces sensitivity to a single noisy validation point. We then restore the checkpoint with the highest primary validation metric observed in that stage and advance to the next stage. The same rule is applied to validation Pass@1 in the final stage for early stopping.

Stage-Aware Probability-Based Token Credit Redistribution Group-based RL assigns a trajectory-level advantage A_i to all generated tokens in trajectory τ_i . The unweighted token contribution is

$$g_{i,j} = A_i \nabla_{\theta} \log \pi_{\theta}(y_{i,j} \mid x_i, y_{i,<j}).$$

DHRCL redistributes this trajectory-level update across tokens:

$$g'_{i,j} = \beta_{i,j} g_{i,j}.$$

Let

$$p_{i,j} = \pi_{\theta_{\text{old}}}(y_{i,j} \mid x_i, y_{i,<j})$$

be the detached rollout-policy probability of token $y_{i,j}$. Token probability is interpreted as how established a decision is under the current policy, not as a token-correctness label.

During the Syntax Stage, we use confidence-oriented weighting,

$$\beta_{i,j}^{\text{conf}} = \frac{p_{i,j}}{\frac{1}{|\tau_i|} \sum_{k=1}^{|\tau_i|} p_{i,k}}.$$

For positive syntax advantages, this consolidates recurrent valid patterns; for negative advantages, it more strongly suppresses recurrent high-probability patterns associated with syntax failure. The goal is to reduce the influence of incidental low-probability samples during early syntax-oriented optimization.

During the Execution Stage, we use

$$\beta_{i,j}^{\text{exec}} = 1.$$

Execution validity frequently depends on non-local interactions, and token probability alone does not reliably localize runtime responsibility. Uniform weighting therefore avoids introducing an unsupported local bias.

During the Pass & Structural Stage, we use uncertainty-oriented weighting,

$$\beta_{i,j}^{\text{unc}} = \frac{1 - p_{i,j}}{\frac{1}{|\tau_i|} \sum_{k=1}^{|\tau_i|} (1 - p_{i,k})}.$$

At this stage, the optimization bottleneck shifts toward decisions that remain less established under the current policy. The weighting does not indiscriminately increase their probability: the sign of A_i determines whether a low-probability decision receives stronger positive credit or stronger negative blame.

All three weighting rules have trajectory-level mean one:

$$\frac{1}{|\tau_i|} \sum_{j=1}^{|\tau_i|} \beta_{i,j} = 1.$$

The mechanism therefore preserves the average contribution of a trajectory while changing how its gradient budget is allocated across generated tokens. Overall, the design follows a consolidation-to-refinement principle: confidence-oriented consolidation in the syntax stage, uniform propagation for non-local execution feedback, and uncertainty-oriented refinement in the final stage. Trajectory rewards are normalized within each prompt-level rollout group before advantage estimation.

Experiments

We evaluate DHRCL under a controlled Qwen3-8B and Kod-Code setting and additionally examine scale robustness using Qwen3-4B, Qwen3-8B, and Qwen3-14B backbones.

Experimental Setup

All trainable methods are initialized from the corresponding backbone checkpoint and trained on KodCode (Xu et al. 2025) under a matched RL protocol. The detailed ablations use Qwen3-8B (Team 2025). The cross-scale comparison uses Qwen3-4B, Qwen3-8B, and Qwen3-14B, which share the same dense Transformer family and tokenizer while differing in capacity. A held-out KodCode validation split is used exclusively for trend-based stage progression, early stopping, and checkpoint selection. Final evaluation uses HumanEval, HumanEval+, BigCodeBench-Full, BigCodeBench-Hard, LiveCodeBench V6, and CodeElo (Chen et al. 2021; Liu et al. 2023a; Zhuo et al. 2025; Jain et al. 2025; Quan et al. 2025). Reference solutions are used only by the AST-based reward and are never included in rollout prompts. All trainable configurations are repeated with three random seeds. Tables report mean $\pm$ standard deviation, and pairwise method differences are additionally assessed with problem-level paired bootstrap confidence intervals.

We compare DHRCL with GRPO, GRPO-PassRate, AceCoder, and VeRPO (Shao et al. 2024; Zeng et al. 2025; Wang et al. 2026b). Each method retains its core reward formulation while using the same policy initialization, training corpus, rollout budget, optimization budget, decoding configuration, and execution sandbox at a given backbone scale.

Pass@1 is the primary functional metric. Syntax Acc. and Exec Acc. describe intermediate program validity. AST Sim., normalized execution time (NET), and normalized test memory usage (NTMU) are complementary diagnostics for accepted outputs. Because NET and NTMU are computed over each method’s own accepted subset, they are descriptive statistics rather than strictly paired efficiency comparisons. For training-process analysis, we report Step Time, GPU Hours, Iterations to Target, Avg. DGR, and Reward Var. Metric definitions and additional implementation details are provided in the Supplementary Document, Sections “Program Validity and Accepted-Output Diagnostics” and “Training-Efficiency and Reward-Signal Metrics.”

Comparison Experiments

Table 1 reports the controlled Qwen3-8B comparison. DHRCL achieves an average Pass@1 of 50.3 ± 0.2 , outperforming VeRPO, the strongest baseline, by 1.1 points. A problem-level paired bootstrap comparison gives a 95% confidence interval of $[0.6, 1.5]$ for this average improvement. The gain is accompanied by higher syntax and execution validity, with Syntax Acc. and Exec Acc. reaching 92.8 ± 0.2 and 82.7 ± 0.3 , respectively. DHRCL also obtains the highest mean AST similarity, while its accepted outputs exhibit lower average NET and NTMU under the adopted evaluation protocol.

Because all baselines are retrained under the unified Qwen3-8B and KodCode protocol, the comparison reflects their adapted performance under matched data, model, and training budgets rather than their originally reported absolute results. Table 3 shows that DHRCL retains the highest average Pass@1 across Qwen3-4B, Qwen3-8B, and Qwen3-14B. Its average improvement over VeRPO is 1.3, 1.1, and

0.7 points, with paired-bootstrap 95% confidence intervals of $[0.8, 1.8]$, $[0.6, 1.5]$, and $[0.3, 1.2]$, respectively. The observed margin narrows with model capacity while remaining positive under the reported paired-bootstrap analysis. Results are not uniformly dominant on every benchmark: for Qwen3-14B, VeRPO is marginally higher on HumanEval, while DHRCL retains larger gains on BigCodeBench-Hard, LiveCodeBench, and CodeElo.

Ablation Studies

This subsection isolates the three main components of DHRCL: reward composition, curriculum scheduling, and probability-aware token credit redistribution. Table 4 summarizes controlled ablations for reward composition, curriculum scheduling, and probability-aware token credit redistribution.

Reward hierarchy. For the reward-component ablation, the curriculum rule, probability-aware token weighting, optimizer, rollout budget, and training budget are held fixed; only the included reward components change. The comparison between *Pass + Syntax + Exec* and *Full Reward* isolates the AST-based structural reward. Adding the AST reward increases average Pass@1 from 49.7 to 50.3 and AST similarity from 0.62 to 0.68. We regard the structural-alignment gain as its primary effect, while the smaller Pass@1 improvement suggests a secondary functional benefit. NET and NTMU are reported as descriptive statistics of accepted outputs because the solved-problem subsets differ across variants.

Curriculum scheduling. All curriculum variants use the same full reward, probability-aware weighting, optimizer, rollout budget, and training budget. They differ only in reward scheduling and stage progression. *Fixed Sum* jointly optimizes all components from the beginning, *Fixed-stage* changes stages after predefined training iterations, and DHRCL uses the recent validation trend. This comparison isolates whether automatically adapting stage duration is preferable to static aggregation or a fixed schedule.

Probability-aware token credit redistribution. All token-weighting variants use the same hierarchical reward and trend-based curriculum. They differ only in how the trajectory-level gradient budget is distributed among generated tokens. No Weighting uses uniform propagation in all stages; Early-only and Late-only apply one side of the proposed ordering; and Reversed applies uncertainty weighting early and confidence weighting late. The full method evaluates the proposed consolidation-to-uniform-to-refinement ordering. The comparison therefore isolates whether stage-dependent token weighting improves the allocation of the trajectory-level gradient budget.

Further Analysis

We use Qwen3-8B for detailed diagnostics because it is the primary ablation setting; cross-scale effectiveness is established in Table 3.

Training dynamics. Figure 2 shows the evolution of held-out validation metrics throughout DHRCL training on

Table 1: Main comparison under the unified Qwen3-8B and KodCode setting. Trainable results are mean $\pm$ standard deviation over three seeds.

Method	Reward	Ext. RM	HE	HE+	BCB-F	BCB-H	LCB	CodeElo	Avg.	Syn. Acc.	Exec Acc.	AST Sim.	NET	NTMU
Qwen3-8B (Team 2025)	–	–	91.0	87.5	35.9	14.5	23.7	16.2	44.8	88.1	74.8	0.55	1.09 $\times$	1.06 $\times$
GRPO (Shao et al. 2024)	0–1	$\times$	92.0 $\pm$ 0.3	88.3 $\pm$ 0.3	36.4 $\pm$ 0.4	16.5 $\pm$ 0.3	27.8 $\pm$ 0.4	26.6 $\pm$ 0.6	47.9 $\pm$ 0.2	89.1 $\pm$ 0.3	77.7 $\pm$ 0.4	0.57 $\pm$ 0.01	1.00 $\pm$ 0.02 $\times$	1.00 $\pm$ 0.02 $\times$
GRPO-PassRate	Pass rate	$\times$	92.4 $\pm$ 0.2	88.5 $\pm$ 0.2	37.0 $\pm$ 0.3	16.9 $\pm$ 0.4	28.6 $\pm$ 0.3	27.0 $\pm$ 0.5	48.4 $\pm$ 0.2	89.5 $\pm$ 0.2	78.7 $\pm$ 0.3	0.59 $\pm$ 0.01	0.97 $\pm$ 0.02 $\times$	0.99 $\pm$ 0.01 $\times$
AceCoder (Zeng et al. 2025)	RM dense	$\checkmark$	92.5 $\pm$ 0.3	88.8 $\pm$ 0.3	36.9 $\pm$ 0.4	16.8 $\pm$ 0.5	28.1 $\pm$ 0.5	27.6 $\pm$ 0.7	48.5 $\pm$ 0.3	89.8 $\pm$ 0.3	79.1 $\pm$ 0.4	0.60 $\pm$ 0.02	0.95 $\pm$ 0.03 $\times$	0.97 $\pm$ 0.02 $\times$
VerPO (Wang et al. 2026b)	Verifiable dense	$\times$	92.8 $\pm$ 0.2	89.2 $\pm$ 0.2	37.6 $\pm$ 0.3	17.6 $\pm$ 0.3	29.4 $\pm$ 0.4	28.7 $\pm$ 0.6	49.2 $\pm$ 0.2	90.5 $\pm$ 0.2	80.3 $\pm$ 0.3	0.63 $\pm$ 0.01	0.92 $\pm$ 0.02 $\times$	0.95 $\pm$ 0.02 $\times$
DHRCL	Hierarchical dense	$\times$	93.1$\pm$0.2	89.8$\pm$0.2	38.3$\pm$0.3	18.6$\pm$0.4	30.5$\pm$0.3	31.4$\pm$0.5	50.3$\pm$0.2	92.8$\pm$0.2	82.7$\pm$0.3	0.68$\pm$0.01	0.85$\pm$0.02$\times$	0.88$\pm$0.02$\times$

Table 2: Training-efficiency and reward-signal statistics on Qwen3-8B. Values are mean $\pm$ standard deviation over three seeds.

Method	Ext. RM	Step Time $\downarrow$	GPU Hours $\downarrow$	Iter. to Target $\downarrow$	Avg. DGR $\downarrow$	Reward Var.
GRPO (Shao et al. 2024)	$\times$	1.00$\pm$0.02$\times$	1.00 $\pm$ 0.04 $\times$	1.00 $\pm$ 0.05 $\times$	0.66 $\pm$ 0.02	0.035 $\pm$ 0.003
GRPO-PassRate	$\times$	1.02 $\pm$ 0.02 $\times$	0.81 $\pm$ 0.04 $\times$	0.79 $\pm$ 0.05 $\times$	0.43 $\pm$ 0.02	0.085 $\pm$ 0.006
AceCoder (Zeng et al. 2025)	$\checkmark$	1.61 $\pm$ 0.05 $\times$	1.34 $\pm$ 0.08 $\times$	0.84 $\pm$ 0.07 $\times$	0.14$\pm$0.02	0.158 $\pm$ 0.011
VerPO (Wang et al. 2026b)	$\times$	1.05 $\pm$ 0.02 $\times$	0.69 $\pm$ 0.04 $\times$	0.66 $\pm$ 0.04 $\times$	0.25 $\pm$ 0.02	0.103 $\pm$ 0.007
DHRCL	$\times$	1.11 $\pm$ 0.03 $\times$	0.55$\pm$0.03$\times$	0.49$\pm$0.04$\times$	<u>0.20$\pm$0.01</u>	0.113 $\pm$ 0.008

Table 3: Cross-scale Pass@1 comparison across Qwen3-4B, Qwen3-8B, and Qwen3-14B. Trainable results are mean $\pm$ standard deviation over three seeds.

Backbone	Method	HE	HE+	BCB-F	BCB-H	LCB	CodeElo	Avg.
Qwen3-4B	Base	86.2	81.9	28.5	10.7	17.6	12.4	39.6
	GRPO	88.5 $\pm$ 0.4	84.4 $\pm$ 0.4	31.5 $\pm$ 0.5	12.8 $\pm$ 0.4	21.6 $\pm$ 0.5	19.8 $\pm$ 0.8	43.1 $\pm$ 0.3
	GRPO-PassRate	88.9 $\pm$ 0.3	84.8 $\pm$ 0.3	31.9 $\pm$ 0.4	13.1 $\pm$ 0.5	22.4 $\pm$ 0.4	20.7 $\pm$ 0.7	43.6 $\pm$ 0.3
	AceCoder	89.2 $\pm$ 0.4	84.9 $\pm$ 0.4	32.3 $\pm$ 0.5	12.9 $\pm$ 0.6	22.0 $\pm$ 0.6	21.2 $\pm$ 0.8	43.8 $\pm$ 0.4
	VerPO	89.6 $\pm$ 0.3	85.4 $\pm$ 0.3	32.9 $\pm$ 0.4	13.8 $\pm$ 0.4	23.2 $\pm$ 0.5	22.0 $\pm$ 0.7	44.5 $\pm$ 0.3
	DHRCL	90.3$\pm$0.3	86.4$\pm$0.3	34.0$\pm$0.4	14.7$\pm$0.5	24.8$\pm$0.4	24.3$\pm$0.6	45.8$\pm$0.3
Qwen3-8B	Base	91.0	87.5	35.9	14.5	23.7	16.2	44.8
	GRPO	92.0 $\pm$ 0.3	88.3 $\pm$ 0.3	36.4 $\pm$ 0.4	16.5 $\pm$ 0.3	27.8 $\pm$ 0.4	26.6 $\pm$ 0.6	47.9 $\pm$ 0.2
	GRPO-PassRate	92.4 $\pm$ 0.2	88.5 $\pm$ 0.2	37.0 $\pm$ 0.3	16.9 $\pm$ 0.4	28.6 $\pm$ 0.3	27.0 $\pm$ 0.5	48.4 $\pm$ 0.2
	AceCoder	92.5 $\pm$ 0.3	88.8 $\pm$ 0.3	36.9 $\pm$ 0.4	16.8 $\pm$ 0.5	28.1 $\pm$ 0.5	27.6 $\pm$ 0.7	48.5 $\pm$ 0.3
	VerPO	92.8 $\pm$ 0.2	89.2 $\pm$ 0.2	37.6 $\pm$ 0.3	17.6 $\pm$ 0.3	29.4 $\pm$ 0.4	28.7 $\pm$ 0.6	49.2 $\pm$ 0.2
	DHRCL	93.1$\pm$0.2	89.8$\pm$0.2	38.3$\pm$0.3	18.6$\pm$0.4	30.5$\pm$0.3	31.4$\pm$0.5	50.3$\pm$0.2
Qwen3-14B	Base	94.5	91.6	43.7	22.8	34.1	28.9	52.6
	GRPO	95.2 $\pm$ 0.2	92.3 $\pm$ 0.2	45.0 $\pm$ 0.3	24.7 $\pm$ 0.4	36.7 $\pm$ 0.4	34.5 $\pm$ 0.6	54.7 $\pm$ 0.2
	GRPO-PassRate	95.5 $\pm$ 0.2	92.5 $\pm$ 0.2	45.5 $\pm$ 0.3	25.1 $\pm$ 0.4	37.1 $\pm$ 0.4	35.0 $\pm$ 0.6	55.1 $\pm$ 0.2
	AceCoder	95.6 $\pm$ 0.2	92.7 $\pm$ 0.2	45.7 $\pm$ 0.4	25.0 $\pm$ 0.5	37.4 $\pm$ 0.5	35.3 $\pm$ 0.7	55.3 $\pm$ 0.3
	VerPO	95.9$\pm$0.2	92.9 $\pm$ 0.2	46.2 $\pm$ 0.3	25.8 $\pm$ 0.4	38.0 $\pm$ 0.4	36.5 $\pm$ 0.6	55.9 $\pm$ 0.2
	DHRCL	95.8 $\pm$ 0.2	93.2$\pm$0.2	46.8$\pm$0.3	26.7$\pm$0.4	38.9$\pm$0.4	38.4$\pm$0.5	56.6$\pm$0.2

Qwen3-8B. Syntax Accuracy improves most rapidly in the first stage, Execution Accuracy rises more clearly after the first transition, and average Pass@1 and AST Similarity

obtain their largest gains after the second transition. Raw measurements fluctuate because of rollout and optimization stochasticity, whereas the moving averages reveal the longer-

Table 4: Controlled ablations on Qwen3-8B. Avg. Pass@1 is reported as mean $\pm$ standard deviation over three seeds.

Group	Variant	Avg.	Syn. Acc.	Exec Acc.	AST Sim.	NET	NTMU	Iter. to Target	Avg. DGR $\downarrow$
Reward	Pass only	48.4 $\pm$ 0.3	89.3	78.5	0.58	0.98 $\times$	0.99 $\times$	0.78 $\times$	0.43
	Pass + Syntax	49.0 $\pm$ 0.3	91.7	79.3	0.60	0.95 $\times$	0.97 $\times$	0.67 $\times$	0.32
	Pass + Exec	48.8 $\pm$ 0.3	89.9	81.7	0.60	0.94 $\times$	0.96 $\times$	0.69 $\times$	0.30
	Pass + Syntax + Exec	49.7 $\pm$ 0.2	92.4	82.1	0.62	0.91 $\times$	0.94 $\times$	0.59 $\times$	0.26
	Full Reward	50.3$\pm$0.2	92.8	82.7	0.68	0.85$\times$	0.88$\times$	0.49$\times$	0.20
Curriculum	Fixed Sum	49.1 $\pm$ 0.3	91.5	80.8	0.64	0.91 $\times$	0.94 $\times$	0.70 $\times$	0.28
	Fixed-stage	49.6 $\pm$ 0.3	92.1	81.9	0.66	0.88 $\times$	0.91 $\times$	0.58 $\times$	0.23
	Trend-based DHRCL	50.3$\pm$0.2	92.8	82.7	0.68	0.85$\times$	0.88$\times$	0.49$\times$	0.20
Prob.-aware	No Weighting	49.4 $\pm$ 0.3	92.0	81.8	0.65	0.90 $\times$	0.92 $\times$	0.64 $\times$	0.25
	Early-only	49.8 $\pm$ 0.2	92.7	82.0	0.66	0.88 $\times$	0.91 $\times$	0.56 $\times$	0.22
	Late-only	49.9 $\pm$ 0.3	92.2	82.5	0.67	0.87 $\times$	0.90 $\times$	0.54 $\times$	0.21
	Reversed	49.2 $\pm$ 0.3	91.6	81.4	0.64	0.92 $\times$	0.94 $\times$	0.68 $\times$	0.27
	Full DHRCL	50.3$\pm$0.2	92.8	82.7	0.68	0.85$\times$	0.88$\times$	0.49$\times$	0.20

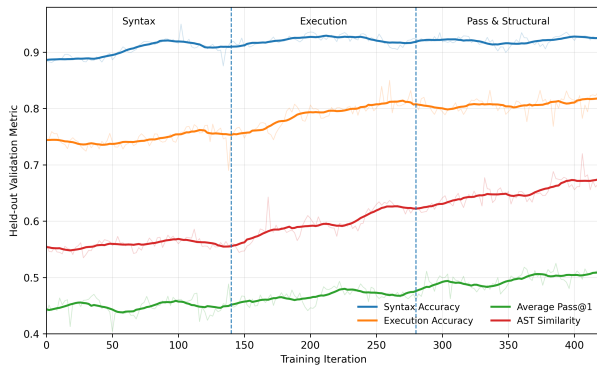


Figure 2: Training dynamics of DHRCL on Qwen3-8B.

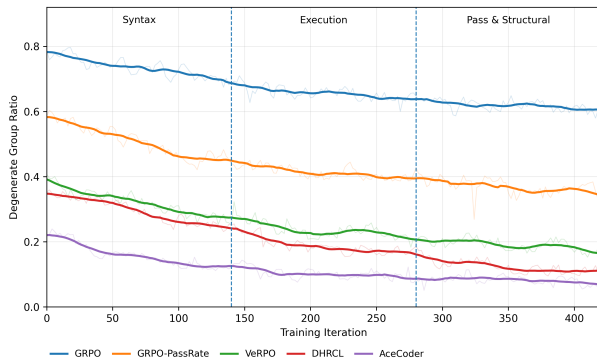


Figure 3: Degenerate group ratio on Qwen3-8B.

term stage-wise progression. Across three seeds, the median transition iterations are 140/280 for Qwen3-8B, 180/340 for Qwen3-4B, and 120/260 for Qwen3-14B, indicating that the selected stage duration varies with model scale.

Reward-signal discriminability. DGR measures how often a rollout group receives nearly identical rewards and therefore little relative-advantage information. As shown in Figure 3, binary GRPO has the highest DGR, partial-test and verifiable rewards reduce it, and AceCoder is lowest because its learned reward model produces continuously varying scores. DHRCL achieves the second-lowest DGR without an external reward model, indicating that its hierarchical feedback distinguishes more candidates within each rollout group than the binary, pass-rate, and verifiable dense-reward alternatives considered here.

Additional error-type breakdowns are provided in the Supplementary Document, Section “Error-Type Analysis.” They show that DHRCL reduces invalid programs while executable-but-incorrect solutions remain the dominant residual failure mode.

Conclusion

We introduced DHRCL, which coordinates syntax, execution, test, and AST-based structural feedback through a trend-adaptive curriculum and stage-aware token credit redistribution. Controlled Qwen3-8B experiments and a Qwen3-4B/8B/14B scale study show consistent gains in functional correctness, intermediate validity, structural alignment, reward discriminability, and convergence efficiency.

Limitations

AST similarity is an auxiliary reference-conditioned signal rather than semantic equivalence; token probability does not identify semantic correctness; the three-stage capability order remains manually specified; and the scale study is limited to the Qwen3 family. NET and NTMU are descriptive statistics over each method’s accepted subset. Broader cross-family and cross-corpus validation remains future work.

Supplementary Material

Overview

This supplementary document provides additional implementation details, evaluation protocols, metric definitions, and extended ablation results.

Baseline Adaptation Protocol

All trainable baselines are retrained under a matched protocol rather than evaluated using released policy checkpoints. At each backbone scale, GRPO, GRPO-PassRate, VerPO, AceCoder, and DHRCL use the same policy initialization, KodCode training split, rollout budget, optimization budget, decoding configuration, execution sandbox, and checkpoint-selection protocol. The reported results therefore characterize these unified adaptations rather than reproducing the original papers’ absolute performance.

Each baseline retains its defining reward formulation. GRPO uses binary complete-test outcomes, GRPO-PassRate uses the fraction of passed test cases, VerPO uses its difficulty-aware verifiable reward and policy objective, and AceCoder uses the released AceCodeRM as an external reward model. Baseline-specific hyperparameters and reward normalization are selected under the same held-out validation protocol and comparable tuning budgets.

Training Configuration

Experiments are conducted on 8 NVIDIA A100 80GB GPUs for the Qwen3-8B setting. The Qwen3-4B experiments use 4 NVIDIA A100 80GB GPUs, while the Qwen3-14B experiments use 16 NVIDIA A100 80GB GPUs. Each rollout batch contains 32 programming problems with 8 candidate solutions per problem. Training-time generation uses temperature $T = 1.0$, top- $p = 1.0$, top- $k = -1$, and a maximum response length of 16384 tokens. Evaluation uses the protocol described below under *Benchmark Evaluation Protocol*.

All methods use the same maximum training-iteration budget at a given backbone scale. Each trainable configuration is run with three random seeds. The main performance results report mean and standard deviation, while supplementary diagnostic tables report seed means unless otherwise stated. All policies are optimized with full-parameter AdamW using a peak learning rate of 1×10^{-6} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, and weight decay 0.1. We use a 3% linear warmup followed by cosine decay, gradient clipping at a global norm of 1.0, one policy-update epoch per rollout batch, bfloat16 mixed precision, and FSDP full sharding. The same optimizer and stabilization settings are used for all trainable methods at a given backbone scale.

For GRPO-style updates, we use Clip-Higher with $\epsilon_{\text{low}} = 0.2$ and $\epsilon_{\text{high}} = 0.28$ and omit an explicit KL penalty. For VerPO, the adapted implementation uses $\alpha = 2.0$ and $\beta = 1.0$; the trajectory-length decay is inactive in the single-turn setting.

For DHRCL, validation is performed every two training iterations on a fixed held-out validation set using a fixed decoding protocol and fixed random seeds. After at least 12 validation observations have been collected in the current stage,

a one-sided linear-trend test at significance level 0.05 is applied to the most recent eight observations. A stage transition is triggered only when the test fails to support a positive trend at two consecutive evaluations. The best checkpoint of the current stage is then restored before training advances. The monitored metric is syntax reward in the Syntax Stage, execution reward in the Execution Stage, and validation Pass@1 in the final stage. All curriculum, reward, and token-weighting ablations use this same rule.

Dataset Splits

We use KodCode as the RL training corpus. The KodCode training split is used for rollout generation, reward computation, and policy optimization. From the training corpus, we reserve a held-out validation split for curriculum-stage switching, early stopping, reward-curve logging, and training-stability analysis. The validation split is never used for policy-gradient updates and is not included in final benchmark reporting.

Final evaluation is conducted only on external benchmark suites, including HumanEval, HumanEval+, BigCodeBench-Full, BigCodeBench-Hard, LiveCodeBench V6, and CodeElo. These benchmarks serve as the test sets for reporting final Pass@1, intermediate validity, and accepted-output diagnostic metrics. This separation ensures that curriculum decisions are made on held-out in-domain validation problems, while final results are measured on external evaluation benchmarks.

Reference solutions in KodCode are used only for computing reward components that require structural comparison, such as AST similarity. They are not included in model inputs and are not exposed to the policy during rollout generation. During rollout generation, the policy only receives the programming problem prompt and generates candidate solutions under the specified decoding configuration.

Benchmark Evaluation Protocol

All methods are evaluated under the same benchmark protocol, decoding configuration, and execution environment. During benchmark evaluation, we sample $n = 8$ candidate solutions for each problem and report Pass@1. Evaluation-time generation uses temperature $T = 0.6$, top- $p = 0.95$, and top- $k = 20$. The maximum response length is set to 16384 tokens for both training and evaluation, following the VerPO-aligned protocol and reducing truncation effects on long-form code-generation tasks. All generated programs are executed in the same sandbox with fixed timeout, memory constraints, and unit-test protocol.

For each benchmark, the same prompt format, decoding parameters, execution sandbox, and pass/fail criteria are used across all methods. For method comparisons, we compute problem-level paired bootstrap confidence intervals with 10,000 resamples, preserving the pairing of predictions on each benchmark problem. This ensures that differences in final results reflect differences in training objectives rather than differences in evaluation configuration. NET and NTMU are computed only on accepted programs. Because each method may solve a different subset of problems, they are interpreted

as descriptive accepted-output diagnostics rather than strictly paired efficiency comparisons.

Program Validity and Accepted-Output Diagnostics

Pass@1 is the primary functional metric. Syntax Acc. is a binary program-level metric obtained with a strict language parser, whereas the training-time syntax reward provides partial credit over local units using an error-tolerant parser. Exec Acc. is the fraction of generated programs that finish without runtime exceptions, timeout, or memory-limit failure.

AST Sim. is computed between a generated program and a verified reference after applying the exact normalization and similarity procedure specified below under *Implementation Details of the AST Structural Reward*. The metric describes reference-conditioned structural alignment and is not interpreted as semantic equivalence. A generated program that cannot be strictly parsed receives zero AST reward during training.

NET is the execution time of an accepted generated program normalized by the corresponding aggregate GRPO value. NTMU is the peak test memory usage of an accepted generated program normalized in the same way. Both metrics are computed over each method’s own accepted outputs. Because the solved-problem subsets can differ, NET and NTMU are descriptive computational characteristics under the adopted evaluation protocol, not strictly paired evidence that one method is universally more efficient.

We keep these evidence levels separate: Pass@1 measures functional correctness; Syntax Acc. and Exec Acc. diagnose intermediate validity; AST Sim. measures structural alignment; and NET/NTMU describe the observed computation of accepted outputs.

Training-Efficiency and Reward-Signal Metrics

We report training-log metrics to compare computational cost, convergence efficiency, reward informativeness, and reward dispersion across RL methods.

Step Time is the average wall-clock time per training iteration, including rollout generation, execution, reward computation, log-probability recomputation, and policy update. GPU Hours measures the GPU-side cost required to reach the target validation performance. It is computed from the average wall-clock time per training iteration, the number of iterations required to reach the target, and the number of GPUs used during training. All values are normalized by GRPO. For methods that do not reach the target within the fixed training budget, we report the GPU Hours consumed under the full budget and mark Iterations to Target as N.R.

Iterations to Target measures how many training iterations are required to reach a fixed validation-performance threshold. We set this threshold to the final validation Pass@1 achieved by GRPO under the same training budget. GRPO therefore serves as the reference method with normalized Iterations to Target equal to $1.00\times$. For methods that do not reach this threshold within the training budget, we report

N.R. A lower value indicates faster convergence to the same validation-performance level.

Avg. DGR denotes the average degenerate group ratio over logged training iterations. For each prompt, the policy samples a rollout group $G = \{\tau_i\}_{i=1}^N$ with scalar rewards $\{R(\tau_i)\}_{i=1}^N$. A rollout group is considered degenerate if

$$\text{Std}(\{R(\tau_i)\}_{i=1}^N) < 10^{-2}.$$

The degenerate group ratio at a training iteration is the proportion of degenerate groups among all prompt-level rollout groups in that iteration. Avg. DGR is the average of this ratio over logged training iterations. A lower Avg. DGR indicates that a larger proportion of rollout groups provide non-trivial relative-advantage signals for policy optimization.

Reward Var. denotes the average within-group reward variance over logged training iterations. It is used to characterize reward dispersion rather than as a metric to be minimized. Extremely low reward variance often indicates uninformative rewards, while excessively high variance may make optimization unstable.

In the reward-signal analysis, DGR is used to compare how often different reward designs fail to distinguish sampled candidates within the same prompt-level rollout group. Binary pass/fail rewards tend to produce high DGR when most sampled programs fail all tests. Pass-rate and difficulty-aware rewards reduce this issue by assigning different scores to partially correct programs. AceCoder achieves the lowest DGR through its external learned reward model. DHRCL achieves the second-lowest DGR without an external reward model because programs may differ in syntax validity, execution status, unit-test pass rate, and structural similarity. This makes the rollout group more likely to provide useful relative-advantage signals during group-based policy optimization.

Implementation Details of the AST Structural Reward

The AST reward is implemented for Python programs using Python 3.11’s built-in `ast` parser. Before comparison, Markdown fences, comments, docstrings, formatting information, and source-location attributes are removed. User-defined identifiers are consistently renamed in order of first occurrence, while operators, control-flow node types, library API names, and the constants -1 , 0 , and 1 are preserved. Other string and numeric literals are mapped to type-level placeholders.

For normalized trees A and B , structural similarity is computed from unit-cost tree-edit distance:

$$\text{Sim}_{\text{AST}}(A, B) = \max\left(0, 1 - \frac{\text{TED}(A, B)}{|A| + |B|}\right),$$

where $|A|$ and $|B|$ denote the numbers of AST nodes. Each problem uses its verified reference solution; when multiple verified references are available, the maximum similarity is used.

For unparsable programs, the training reward is zero. For parsable programs, the returned similarity is bounded in $[0, 1]$. This signal is used only as reference-conditioned

Table 5: Error-type distribution under the unified Qwen3-8B evaluation protocol. All categories are mutually exclusive, and each row sums to 100%.

Method	SyntaxError (%) ↓	RuntimeError (%) ↓	Timeout (%) ↓	Wrong Answer (%) ↓	Accepted (%) ↑
Qwen3-8B (Team 2025)	8.0	9.7	4.8	32.7	44.8
GRPO (Shao et al. 2024)	7.3	8.7	4.1	32.0	47.9
GRPO-PassRate	6.8	8.1	3.8	32.9	48.4
AceCoder (Zeng et al. 2025)	7.1	8.3	4.0	32.1	48.5
VerPO (Wang et al. 2026b)	6.2	7.4	3.5	33.7	49.2
DHRCL	4.8	5.9	2.9	36.1	50.3

Table 6: Reward-component ablation.

Variant	Syn.	Exec	Pass	AST	HE	HE+	BCB-F	BCB-H	LCB	CodeElo	Avg.	Syn. Acc.	Exec Acc.	AST Sim.	NET	NTMU
Pass only	✗	✗	✓	✗	92.3	88.5	36.9	16.8	28.4	27.5	48.4	89.3	78.5	0.58	0.98×	0.99×
Pass + Syntax	✓	✗	✓	✗	92.7	89.0	37.3	17.3	29.0	28.7	49.0	91.7	79.3	0.60	0.95×	0.97×
Pass + Exec	✗	✓	✓	✗	92.5	88.8	37.1	17.2	29.0	27.9	48.8	89.9	81.7	0.60	0.94×	0.96×
Pass + Syntax + Exec	✓	✓	✓	✗	92.8	89.3	37.7	18.0	29.8	30.6	49.7	92.4	82.1	0.62	0.91×	0.94×
Full Reward	✓	✓	✓	✓	93.1	89.8	38.3	18.6	30.5	31.4	50.3	92.8	82.7	0.68	0.85×	0.88×

Table 7: Curriculum and token-credit ablations.

Group	Variant	Avg. Pass@1	Iterations to Target	Avg. DGR	GPU Hours	Stage Transitions
Curriculum	Fixed Sum	49.1±0.3	0.70×	0.28	0.78×	—
Curriculum	Fixed-stage	49.6±0.3	0.58×	0.23	0.65×	160/300
Curriculum	Trend-based DHRCL	50.3±0.2	0.49×	0.20	0.55×	140/280
Token	No Weighting	49.4±0.3	0.64×	0.25	0.72×	160/300
Token	Early-only	49.8±0.2	0.56×	0.22	0.63×	140/300
Token	Late-only	49.9±0.3	0.54×	0.21	0.61×	160/280
Token	Reversed	49.2±0.3	0.68×	0.27	0.76×	180/320
Token	Full DHRCL	50.3±0.2	0.49×	0.20	0.55×	140/280

structural supervision; unit-test feedback remains the direct functional objective.

Implementation Details of Syntax Reward

During RL training, generated programs often contain local syntax errors, so a strict AST parser may provide no partial feedback. We therefore use Tree-sitter as an error-tolerant parser to obtain partial syntax trees after removing Markdown code fences and normalizing indentation when possible.

We extract statement- and block-level syntactic units from the parse tree, including function definitions, class definitions, control-flow statements, assignments, expressions, and return statements. Lexical leaves, comments, and overly short fragments are excluded from reward computation. A unit is marked as syntactically valid if its subtree does not contain parser-reported error nodes. The final syntax reward is the length-weighted fraction of valid units:

$$r_{\text{syntax}}(y) = \frac{\sum_{i=1}^m w_i v_i}{\sum_{i=1}^m w_i}.$$

We use token length as w_i so that larger program regions contribute proportionally more to the reward. If the parser cannot extract any statement-level unit, the syntax reward is set to zero.

This implementation makes the training reward robust to early-stage generations that are only partially well-formed. It also distinguishes the training-time syntax reward from the evaluation-time Syntax Acc. metric: the former gives partial credit over local syntactic units, whereas the latter checks whether the complete generated program is syntactically valid.

Error-Type Analysis

To complement the aggregate benchmark scores in the main paper, we categorize each generated program by its final execution outcome. The five categories are mutually exclusive: a sample is assigned to the first applicable category among syntax error, runtime error, timeout, wrong answer, and accepted. Consequently, every row in Table 5 sums to 100%. For each seed, the error-type percentages are computed separately for each benchmark and then macro-averaged across the six benchmarks, assigning equal weight to each benchmark. The reported values are subsequently averaged over three seeds. Under this aggregation, the Accepted proportion corresponds to the average Pass@1 reported in the main paper.

Compared with the base model, DHRCL reduces syntax errors from 8.0% to 4.8%, runtime errors from 9.7% to 5.9%, and timeouts from 4.8% to 2.9%. Relative to VerPO, DHRCL also lowers all three invalid-program categories and increases the accepted proportion from 49.2% to 50.3%. Executable-but-incorrect programs remain the dominant residual failure mode after syntax and execution failures are reduced.

Detailed Ablation Results

Table 6 reports the benchmark-level reward-component ablation, while Table 7 summarizes the curriculum-scheduling and token-credit ablations. Stage Transitions report the median first/second transition iterations over three seeds. Together, these results isolate the contributions of reward composition, trend-based stage progression, and stage-aware probability-based token credit redistribution.

Reward components. As shown in Table 6, adding syntax and execution feedback improves intermediate program validity, while the full reward obtains the highest average Pass@1 and AST similarity. The comparison between *Pass + Syntax + Exec* and *Full Reward* associates the AST component with a 0.6-point increase in average Pass@1 and a clearer improvement in AST similarity. We therefore interpret structural alignment as the primary direct effect and the Pass@1 gain as a possible secondary benefit.

Curriculum and token credit. Table 7 shows that trend-based stage progression outperforms both fixed reward aggregation and predefined stage switching. Among token-credit variants, the complete consolidation-to-refinement ordering achieves the highest average Pass@1, the lowest DGR, and the smallest normalized training cost. The reversed variant performs worse than either the early-only or late-only variants, supporting the importance of the proposed stage ordering.

References

- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H. P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F. P.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W. H.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A. N.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code. *CoRR*, abs/2107.03374.
- Dai, N.; Wu, Z.; Zheng, R.; Wei, Z.; Shi, W.; Jin, X.; Liu, G.; Dun, C.; Huang, L.; and Yan, L. 2024. Process Supervision-Guided Policy Optimization for Code Generation. *CoRR*, abs/2410.17621.
- DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR*, abs/2501.12948.
- Dou, S.; Liu, Y.; Jia, H.; Xiong, L.; Zhou, E.; Shen, W.; Shan, J.; Huang, C.; Wang, X.; Fan, X.; Xi, Z.; Zhou, Y.; Ji, T.; Zheng, R.; Zhang, Q.; Huang, X.; and Gui, T. 2024. StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback. *CoRR*, abs/2402.01391.
- Gehring, J.; Zheng, K.; Copet, J.; Mella, V.; Cohen, T.; and Synnaeve, G. 2025. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. In Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; and Zhu, J., eds., *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net.
- Jain, N.; Han, K.; Gu, A.; Li, W.; Yan, F.; Zhang, T.; Wang, S.; Solar-Lezama, A.; Sen, K.; and Stoica, I. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.
- Jiang, J.; Wang, F.; Shen, J.; Kim, S.; and Kim, S. H. 2026. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.*, 35(2): 58:1–58:72.
- Le, H.; Wang, Y.; Gotmare, A. D.; Savarese, S.; and Hoi, S. C. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- Liu, J.; Xia, C. S.; Wang, Y.; and Zhang, L. 2023a. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation.
- Liu, J.; Zhu, Y.; Xiao, K.; Fu, Q.; Han, X.; Yang, W.; and Ye, D. 2023b. RLTF: Reinforcement Learning from Unit Test Feedback. *Trans. Mach. Learn. Res.*, 2023.
- Nair, M.; Yamani, K. M.; L’Hadj, L. S.; and Baghdadi, R. 2024. Curriculum Learning for Small Code Language Models. In Fu, X.; and Fleisig, E., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, 531–542. Association for Computational Linguistics.
- Park, C.; Jiang, J.; Wang, F.; Paul, S.; Shen, J.; Tang, J.; and Li, J. 2026. TAROT: Test-driven and Capability-adaptive Curriculum Reinforcement Fine-tuning for Code Generation with Large Language Models. *CoRR*, abs/2602.15449.
- Quan, S.; Yang, J.; Yu, B.; Zheng, B.; Liu, D.; Yang, A.; Ren, X.; Gao, B.; Miao, Y.; Feng, Y.; Wang, Z.; Yang, J.; Cui, Z.; Fan, Y.; Zhang, Y.; Hui, B.; and Lin, J. 2025. CodeElo: Benchmarking Competition-level Code Generation of LLMs with Human-comparable Elo Ratings. *CoRR*, abs/2501.01257.
- Rafailov, R.; Sharma, A.; Mitchell, E.; Manning, C. D.; Ermon, S.; and Finn, C. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Ren, H.; Lu, Z.; Shi, W.; Hou, H.; Yang, Y.; Wang, K.; Zhou, A.; Pan, J.; Zhan, M.; and Li, H. 2025. Alignment with Fill-In-the-Middle for Enhancing Code Generation. In Christodoulopoulos, C.; Chakraborty, T.; Rose, C.; and Peng, V., eds., *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, 8304–8320. Association for Computational Linguistics.
- Rozière, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X. E.; Adi, Y.; Liu, J.; Remez, T.; Rapin, J.; Kozhevnikov, A.; Evtimov, I.; Bitton, J.; Bhatt, M.; Canton-Ferrer, C.; Grattafiori, A.; Xiong, W.; Défossez, A.; Copet, J.; Azhar, F.; Touvron, H.; Martin, L.; Usunier, N.; Scialom, T.; and

- Synnaeve, G. 2023. Code Llama: Open Foundation Models for Code. *CoRR*, abs/2308.12950.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *CoRR*, abs/1707.06347.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *CoRR*, abs/2402.03300.
- Shojaee, P.; Jain, A.; Tipirneni, S.; and Reddy, C. K. 2023. Execution-based Code Generation using Deep Reinforcement Learning. *Trans. Mach. Learn. Res.*, 2023.
- Team, Q. 2025. Qwen3 Technical Report. *CoRR*, abs/2505.09388.
- Wang, A.; Yan, Y.; Zhou, N.; Lu, Z.; Lu, W.; Xiao, J.; Zhuang, Y.; and Shen, Y. 2026a. Code-A1: Adversarial Evolving of Code LLM and Test LLM via Reinforcement Learning. *CoRR*, abs/2603.15611.
- Wang, J.; Zhang, Z.; He, Y.; Song, Y.; Shi, T.; Li, Y.; Xu, H.; Wu, K.; Qian, G.; Chen, Q.; and He, L. 2024. Enhancing Code LLMs with Reinforcement Learning in Code Generation: A Survey. *CoRR*, abs/2412.20367.
- Wang, L.; Wu, X.; Hu, X.; Liu, Y.; Fan, Y.; Yu, K.; Weng, Q.; Xi, W.; and Li, X. 2026b. VerPO: Verifiable Dense Reward Policy Optimization for Code Generation. *CoRR*, abs/2601.03525.
- Wang, Z.; Zhou, S.; Fried, D.; and Neubig, G. 2023. Execution-Based Evaluation for Open-Domain Code Generation. In Bouamor, H.; Pino, J.; and Bali, K., eds., *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, Findings of ACL, 1271–1290. Association for Computational Linguistics.
- Williams, R. J. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Mach. Learn.*, 8: 229–256.
- Wu, H.; Yao, Y.; Yu, W.; and Zhang, N. 2026. ReCode: Updating Code API Knowledge with Reinforcement Learning. In Koenig, S.; Jenkins, C.; and Taylor, M. E., eds., *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, 33908–33916. AAAI Press.
- Wu, J.; Li, H.; Zhang, X.; Liu, X.; Huang, Y.; Luo, J.; Zhang, Y.; Li, Z.; Chu, R.; Yang, Y.; and Li, S. 2025. Teaching Your Models to Understand Code via Focal Preference Alignment. In Christodoulopoulos, C.; Chakraborty, T.; Rose, C.; and Peng, V., eds., *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, 14003–14023. Association for Computational Linguistics.
- Xu, Z.; Liu, Y.; Yin, Y.; Zhou, M.; and Poovendran, R. 2025. KodCode: A Diverse, Challenging, and Verifiable Synthetic Dataset for Coding. 6980–7008.
- Ye, Y.; Zhang, T.; Jiang, W.; and Huang, H. 2025. Process-Supervised Reinforcement Learning for Code Generation. In Christodoulopoulos, C.; Chakraborty, T.; Rose, C.; and Peng, V., eds., *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, 14213–14226. Association for Computational Linguistics.
- Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Fan, T.; Liu, G.; Liu, L.; Liu, X.; Lin, H.; Lin, Z.; Ma, B.; Sheng, G.; Tong, Y.; Zhang, C.; Zhang, M.; Zhang, W.; Zhu, H.; Zhu, J.; Chen, J.; Chen, J.; Wang, C.; Yu, H.; Dai, W.; Song, Y.; Wei, X.; Zhou, H.; Liu, J.; Ma, W.; Zhang, Y.; Yan, L.; Qiao, M.; Wu, Y.; and Wang, M. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *CoRR*, abs/2503.14476.
- Zeng, H.; Jiang, D.; Wang, H.; Nie, P.; Chen, X.; and Chen, W. 2025. ACECODER: Acing Coder RL via Automated Test-Case Synthesis. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds., *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, 12023–12040. Association for Computational Linguistics.
- Zhang, D.; Diao, S.; Zou, X.; and Peng, H. 2024. PLUM: Preference Learning Plus Test Cases Yields Better Code Language Models. *CoRR*, abs/2406.06887.
- Zhang, K.; Li, G.; Dong, Y.; Xu, J.; Zhang, J.; Su, J.; Liu, Y.; and Jin, Z. 2025a. CodeDPO: Aligning Code Models with Self Generated and Verified Source Code. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds., *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, 15854–15871. Association for Computational Linguistics.
- Zhang, K.; Li, G.; Li, J.; Dong, Y.; and Jin, Z. 2025b. Focused-DPO: Enhancing Code Generation Through Focused Preference Optimization on Error-Prone Points. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds., *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Findings of ACL, 9578–9591. Association for Computational Linguistics.
- Zhao, Y.; Fu, W.; Li, S.; Hu, Y.; Guo, X.; and Jin, Y. 2025. Enhancing LLM Performance on Hardware Design Generation Task via Reinforcement Learning. In *IEEE International Symposium on Circuits and Systems, ISCAS 2025, London, United Kingdom, May 25-28, 2025*, 1–5. IEEE.
- Zheng, C.; Liu, S.; Li, M.; Chen, X.; Yu, B.; Gao, C.; Dang, K.; Liu, Y.; Men, R.; Yang, A.; Zhou, J.; and Lin, J. 2025. Group Sequence Policy Optimization. *CoRR*, abs/2507.18071.
- Zhuo, T. Y.; Vu, M. C.; Chim, J.; Hu, H.; Yu, W.; Widyasari, R.; Yusuf, I. N. B.; Zhan, H.; He, J.; Paul, I.; Brunner, S.; Gong, C.; Hoang, J.; Zebaze, A. R.; Hong, X.; Li, W.; Kadour, J.; Xu, M.; Zhang, Z.; Yadav, P.; and et al. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.